

Implement Client Script & UI Policy (Incident)

Dynamic form control on the ServiceNow Incident form

Project Description:

This project implements **Client Scripts** and **UI Policies** on the ServiceNow **Incident** form to improve user experience, enforce field behaviour and reduce incorrect data entry. The form controls fields dynamically, based on user actions and incident conditions, without needing the form to be submitted.

The solution includes a UI Policy on the Incident form, a UI Policy Action that controls the Urgency field, an onChange Client Script that reacts to field changes, an onSubmit Client Script that validates data before saving, and an onCellEdit Client Script that stops the State field from being changed through list editing. Everything is built on a ServiceNow Personal Developer Instance using JavaScript and the platform's own form tools, so no extra servers or databases are needed.

Team ID: SWTID-2026-8305 | Team: Varsha Devi K H (Team Lead), Varshida Kannivadi Balaji, Nithishkumar N K, Yogasairam P

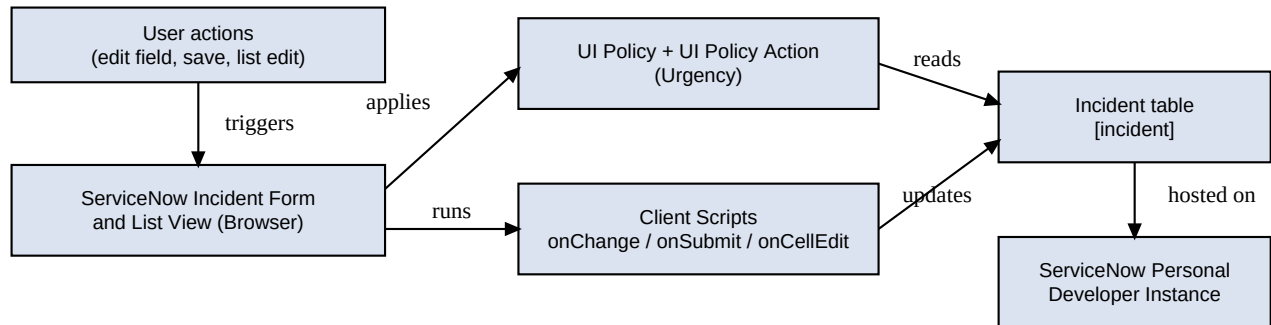
Scenarios

Scenario 1: A support agent sets the Impact of an incident to High. The UI Policy shows the message "Assigned To is mandatory for High impact incidents.", the Urgency field becomes read-only and Priority is calculated as 1 - Critical, all instantly on the form.

Scenario 2: The agent changes the Impact back to Medium. Urgency becomes editable again and Priority changes to 2 - High, so the form returns to its normal behaviour without saving or reloading.

Scenario 3: A user tries to edit the State of an incident directly in the Incident list view. The onCellEdit Client Script blocks the change and shows "State cannot be updated using list editing. Please open the Incident.", so State is only changed from the full form.

Technical Architecture:



Description: The project uses a simple three-part setup. The user works in the Incident form or list view in the browser. UI Policies and UI Policy Actions control field behaviour (mandatory, visible, read-only) based on conditions, while Client Scripts run JavaScript when a field changes, when the form is saved, or when a list cell is edited. Both run in the browser, so they respond instantly, and they work on the existing Incident table hosted on a ServiceNow Personal Developer Instance.

(Note: This project uses no custom database. It works on ServiceNow's built-in Incident table, so no ER diagram is needed.)

Pre-requisites:

- **ServiceNow Personal Developer Instance (PDI):** <https://developer.servicenow.com>
- **Incident Management enabled:** Incident application and form access
- **Admin or developer role:** to create Client Scripts and UI Policies
- **JavaScript knowledge:** basic JavaScript and ServiceNow client APIs (g_form)
- **Modern browser:** Google Chrome, Microsoft Edge or Firefox
- **ServiceNow documentation:** <https://docs.servicenow.com>

Project Workflow:

Milestone 1: Create UI Policy on Incident

- **Activity 1.1:** Set up the ServiceNow instance and open the Incident form.
- **Activity 1.2:** Create a UI Policy on the Incident table with conditions and actions.

Milestone 2: Create UI Policy Action – Urgency

- **Activity 2.1:** Add a UI Policy Action that controls the Urgency field when the condition is met.

Milestone 3: Create onChange Client Script

- **Activity 3.1:** Write an onChange Client Script on the Incident table that reacts when a field value changes.

Milestone 4: Create onSubmit Client Script (Save Validation)

- **Activity 4.1:** Write an onSubmit Client Script that validates the form and blocks an invalid save.

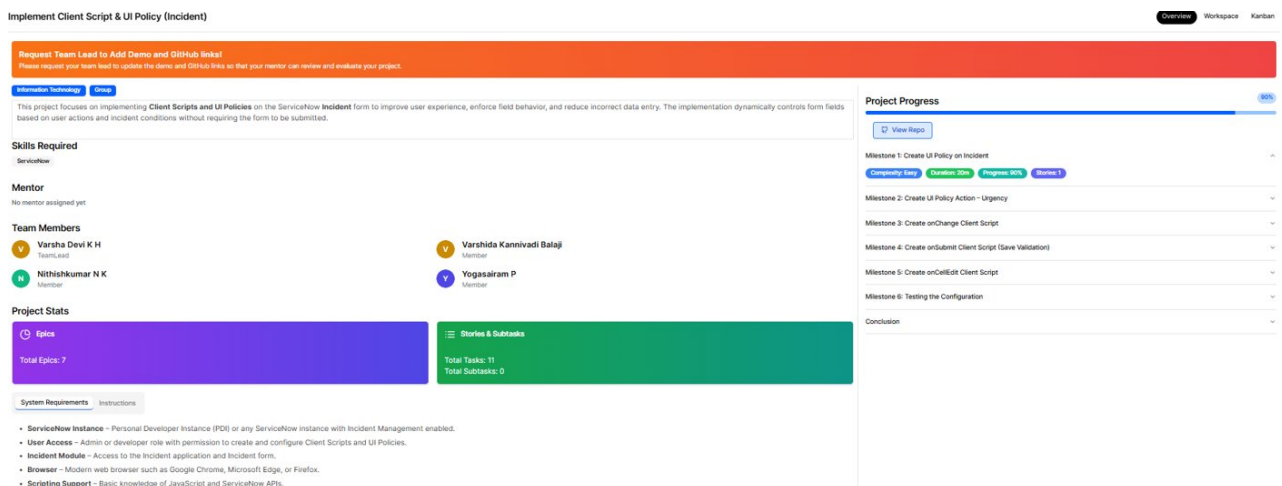
Milestone 5: Create onCellEdit Client Script

- **Activity 5.1:** Write an onCellEdit Client Script that stops State from being changed through list editing.

Milestone 6: Testing the Configuration

- **Activity 6.1:** Test every UI Policy and Client Script on the Incident form and list view.

Milestone 7: Conclusion



Implement Client Script & UI Policy (Incident)

Request Team Lead to Add Demo and GitHub links!
Please request your team lead to update the demo and GitHub links so that your mentor can review and evaluate your project.

System Requirements [View Repo](#)

This project focuses on implementing Client Scripts and UI Policies on the ServiceNow Incident form to improve user experience, enforce field behavior, and reduce incorrect data entry. The implementation dynamically controls form fields based on user actions and incident conditions without requiring the form to be submitted.

Skills Required
ServiceNow

Mentor
No mentor assigned yet.

Team Members

Name	Role
Varsha Devi K H	Team Lead
Nithishkumar N K	Member
Varshida Kannivadi Balaji	Member
Yogasairam P	Member

Project Stats

Epics	Stories & Subtasks
Total Epics: 7	Total Tasks: 11 Total Subtasks: 0

System Requirements [Instructions](#)

- **ServiceNow Instance** - Personal Developer Instance (PDI) or any ServiceNow Instance with Incident Management enabled.
- **User Access** - Admin or developer role with permission to create and configure Client Scripts and UI Policies.
- **Incident Module** - Access to the Incident application and Incident form.
- **Browser** - Modern web browser such as Google Chrome, Microsoft Edge, or Firefox.
- **Scripting Support** - Basic knowledge of JavaScript and ServiceNow APIs.

Project Progress [View Repo](#)

100%

- Milestone 1: Create UI Policy on Incident [Completed](#) [View Repo](#) [Progress 100%](#) [Stories 1](#)
- Milestone 2: Create UI Policy Action - Urgency
- Milestone 3: Create onChange Client Script
- Milestone 4: Create onSubmit Client Script (Save Validation)
- Milestone 5: Create onCellEdit Client Script
- Milestone 6: Testing the Configuration
- Conclusion

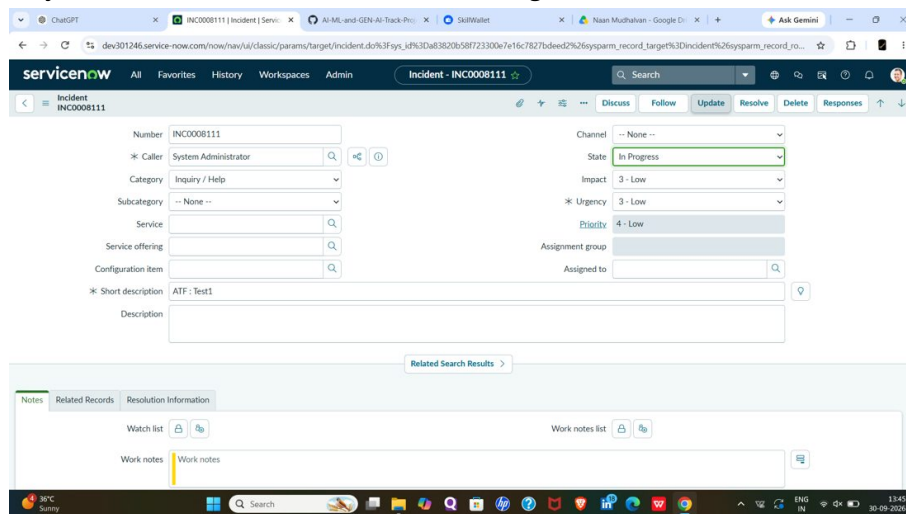
Project overview page showing team members, milestones and progress

Milestone 1: Create UI Policy on Incident

In this milestone, we prepare the instance and create a UI Policy on the Incident form. A UI Policy changes how fields look and behave (mandatory, visible, read-only) as soon as a condition is true, without any scripting needed to save the form.

Activity 1.1: Set Up the ServiceNow Instance

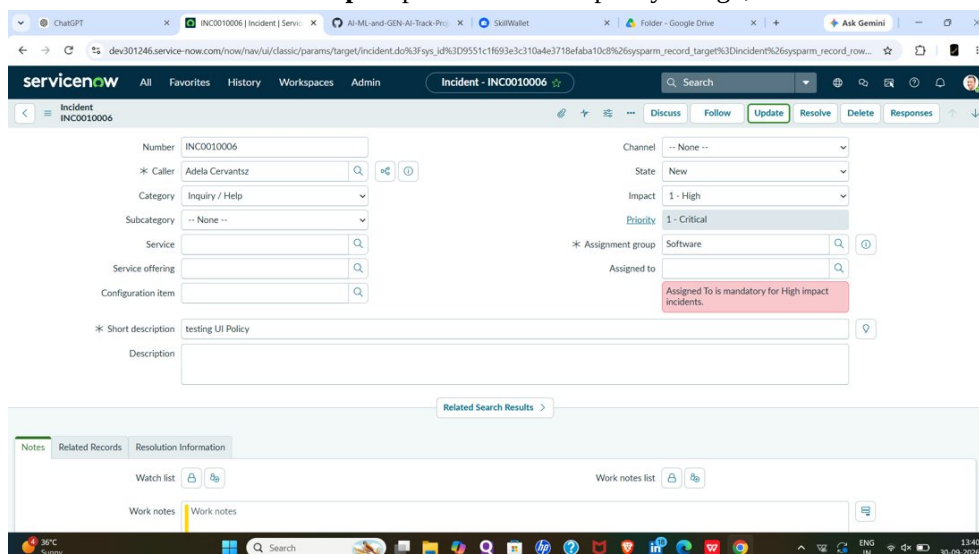
- **Step 1:** Sign up for a free Personal Developer Instance at developer.servicenow.com and log in as an administrator.
- **Step 2:** Confirm that Incident Management is available by opening **Incident > All** from the application navigator.
- **Step 3:** Open any incident to see the form that will be configured.



Incident form (INC0008111) that the UI Policy and Client Scripts are applied to

Activity 1.2: Create the UI Policy

- **Step 1:** Go to **System UI > UI Policies** and click **New**.
- **Step 2:** Select the table **Incident [incident]** and give the policy a clear short description.
- **Step 3:** Set the condition so the policy applies when **Impact** is High.
- **Step 4:** Add the actions so the form shows the message "Assigned To is mandatory for High impact incidents." and flags the required assignment field.
- **Step 5:** Make sure **Active** and **Run scripts** options match the policy design, then save.



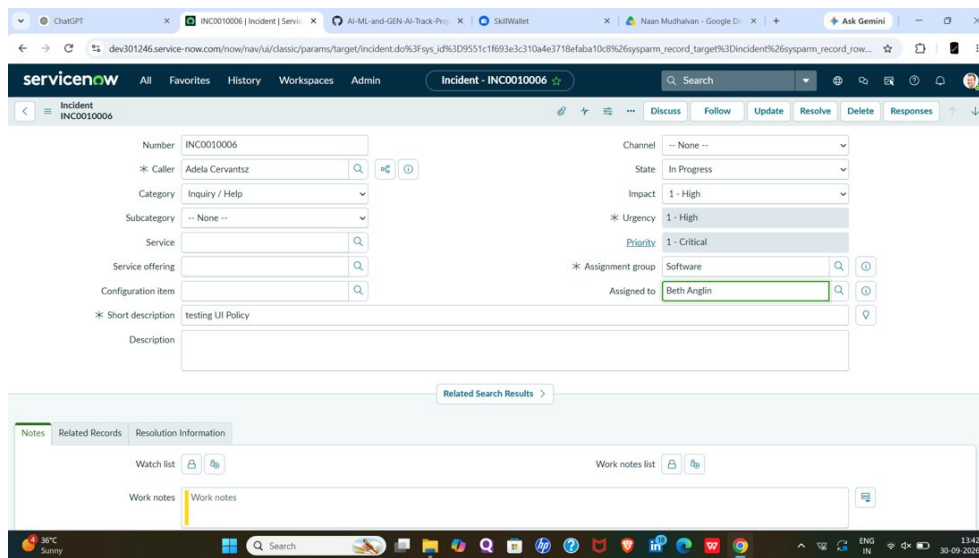
UI Policy at work: Impact = High shows the mandatory message on the Incident form

Milestone 2: Create UI Policy Action – Urgency

A UI Policy Action is the part of a UI Policy that decides what happens to a single field. In this milestone, an action is added for the **Urgency** field so that it is controlled by the Impact value.

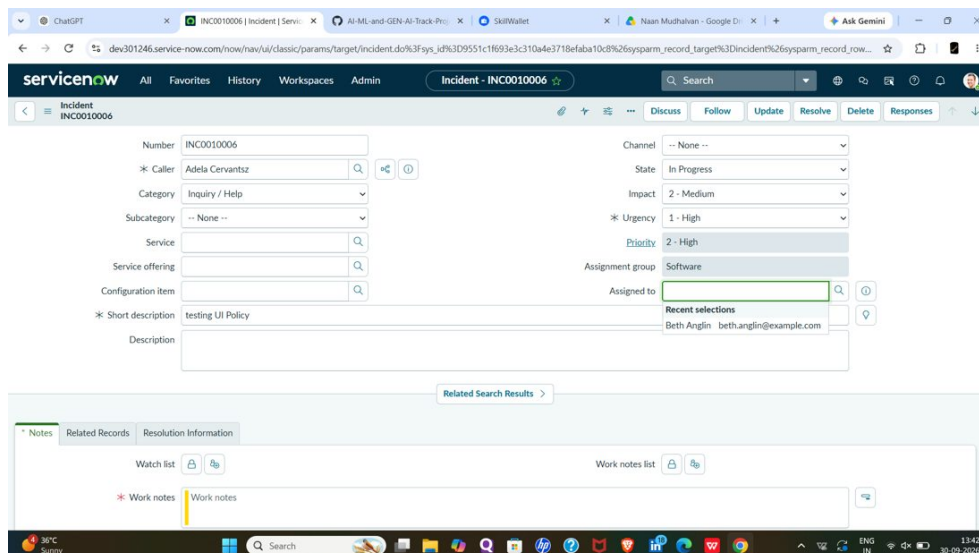
Activity 2.1: Add the Urgency Action

- **Step 1:** Open the UI Policy and scroll to the **UI Policy Actions** related list.
- **Step 2:** Click **New** and choose the field **Urgency**.
- **Step 3:** Set the field to **Read only: True** (Mandatory and Visible stay as designed) and save the action.
- **Step 4:** Open an incident, set Impact to High and check that Urgency is greyed out and Priority shows 1 - Critical.



Impact = High: Urgency is read-only and Priority is 1 - Critical

- **Step 5:** Change Impact to Medium and check that Urgency becomes editable again and Priority changes to 2 - High.



Impact = Medium: Urgency is editable again and Priority is 2 - High

Milestone 3: Create onChange Client Script

An **onChange** Client Script runs in the browser the moment a chosen field value changes. It can update other fields or show a message straight away, without saving the form.

Activity 3.1: Create the onChange Script

- **Step 1:** Go to **System Definition > Client Scripts** and click **New**.
- **Step 2:** Enter a descriptive name, select the table **Incident [incident]**, UI Type **Desktop** and Type **onChange**.
- **Step 3:** Choose the **Field name** to watch and tick **Active**.
- **Step 4:** Write the script using the standard onChange function and the g_form API, then save.
- **Step 5:** Open an incident, change the watched field and check that the form reacts immediately.

Milestone 4: Create onSubmit Client Script (Save Validation)

An **onSubmit** Client Script runs when the user saves the form. If the data is not valid, the script stops the save and shows a clear error message, which reduces incorrect data entry.

Activity 4.1: Create the onSubmit Script

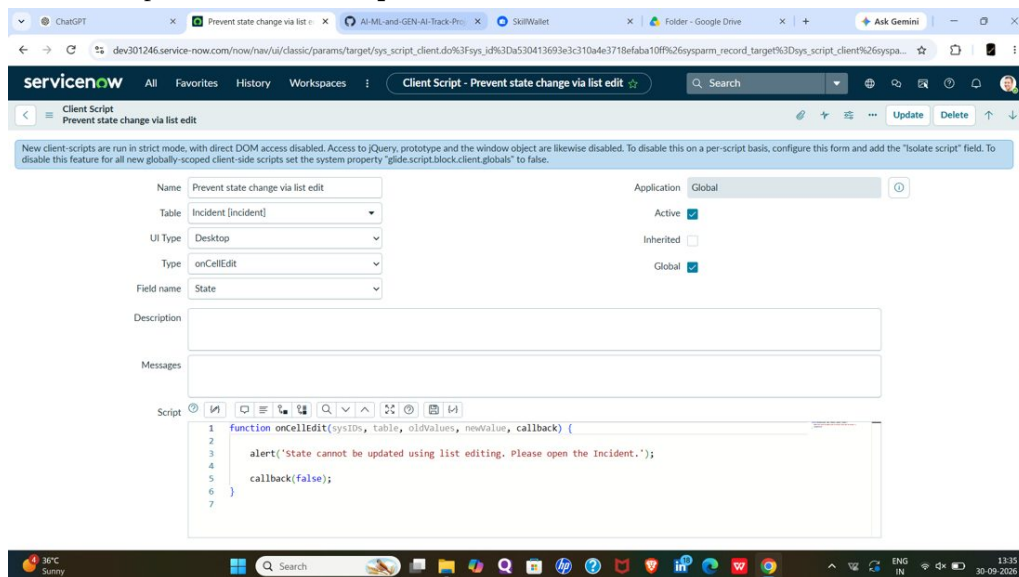
- **Step 1:** Go to **System Definition > Client Scripts** and click **New**.
- **Step 2:** Select the table **Incident [incident]**, UI Type **Desktop** and Type **onSubmit**.
- **Step 3:** Write the script to check the required values and return **false** with an error message when they are not valid.
- **Step 4:** Save the script, then try to save an incident with invalid or missing data and check that the save is blocked.

Milestone 5: Create onCellEdit Client Script

An **onCellEdit** Client Script runs when a user edits a cell directly in a list view. In this project it is used to stop the **State** of an incident from being changed through list editing, so State is only updated from the full Incident form.

Activity 5.1: Create the onCellEdit Script

- **Step 1:** Go to **System Definition > Client Scripts** and click **New**.
- **Step 2:** Fill in the form as shown below: Name **Prevent state change via list edit**, Table **Incident [incident]**, UI Type **Desktop**, Type **onCellEdit**, Field name **State**, Application **Global**, and tick **Active**.
- **Step 3:** Add the script below and click **Update**.



Client Script record: Prevent state change via list edit

```
function onCellEdit(sysIDs, table, oldValues, newValue, callback) {
    alert('State cannot be updated using list editing. Please open the Incident. ');
    callback(false);
}
```

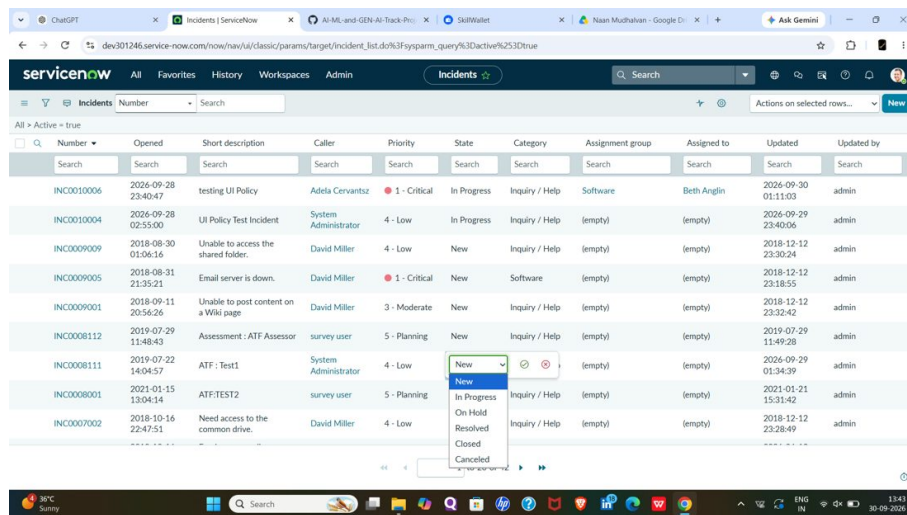
How it works: the script shows an alert to the user, and **callback(false)** tells ServiceNow to reject the edit, so the State value in the list does not change.

Milestone 6: Testing the Configuration

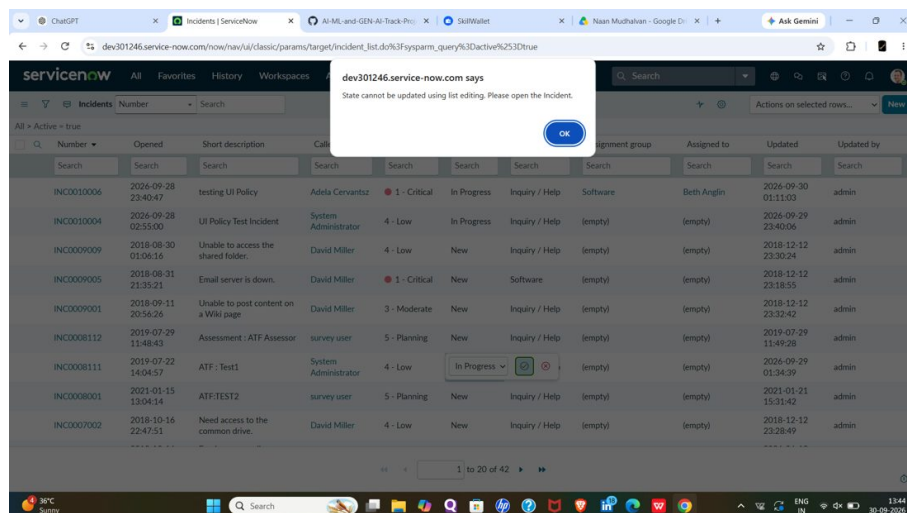
In this milestone, every UI Policy and Client Script is tested on the Incident form and list view to confirm it behaves as designed.

Activity 6.1: Test the Scenarios

S.No	Test	Expected result	Result
1	On a High impact incident, leave Assigned to empty.	Message "Assigned To is mandatory for High impact incidents." is shown.	Pass
2	Set Impact to High.	Urgency becomes read-only and Priority shows 1 - Critical.	Pass
3	Change Impact from High to Medium.	Urgency becomes editable and Priority changes to 2 - High.	Pass
4	Edit the State cell in the Incident list view.	Alert is shown and State is not changed.	Pass



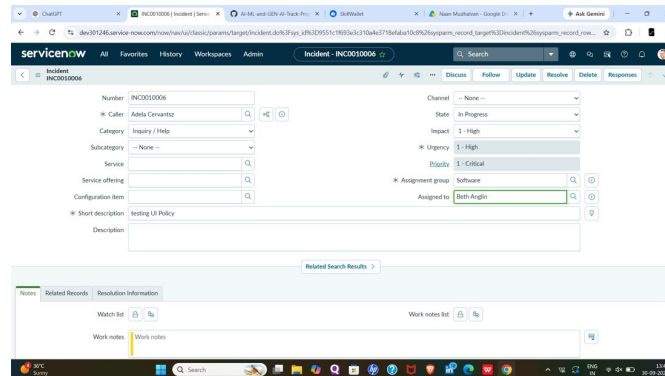
List view: the State cell editor opened on INC0008111



The onCellEdit alert: State cannot be updated using list editing

Exploring the Project Screens:

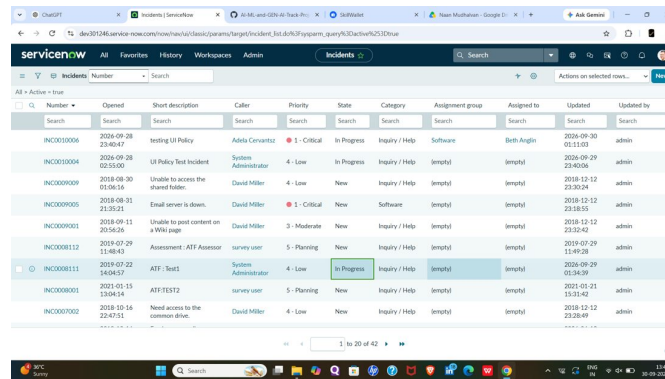
Incident Form:



The screenshot shows the ServiceNow Incident Form for incident INC0010006. The form is divided into several sections. The top section contains fields for Number (INC0010006), Caller (Adela Cervantes), Category (Inquiry / Help), Subcategory (None), Service, Service offering, Configuration item, and Description. The right side of the form contains dropdown menus for Channel (None), State (In Progress), Impact (1 - High), Urgency (1 - High), Priority (1 - Critical), Assignment group (Software), and Assigned to (Beth Anglin). Below the form, there is a 'Related Search Results' section with tabs for 'None', 'Related Records', and 'Resolution Information'. The 'Related Records' tab is currently selected, showing a 'Watch list' and 'Work notes' section.

Description: The Incident form is where the UI Policy and UI Policy Action work. When Impact is High, the form shows the mandatory message, locks the Urgency field and updates Priority. When Impact changes, the form returns to normal behaviour on its own.

Incident List View:

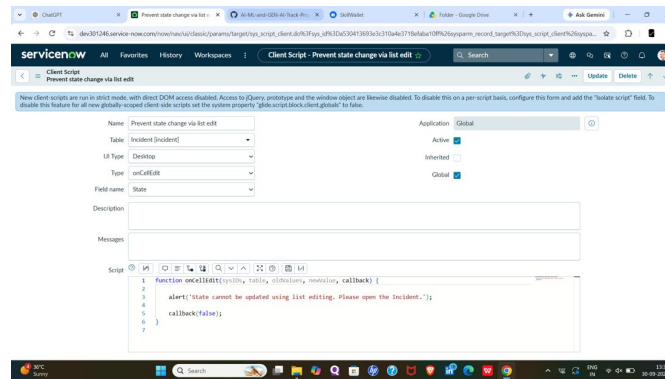


The screenshot shows the ServiceNow Incident List View. The table displays a list of incidents with columns for Number, Opened, Short description, Caller, Priority, State, Category, Assignment group, Assigned to, Updated, and Updated by. The 'State' column for the selected incident (INC0000111) is highlighted in green and labeled 'In Progress'.

Number	Opened	Short description	Caller	Priority	State	Category	Assignment group	Assigned to	Updated	Updated by
INC0000006	2026-09-28 23:40:47	testing UI Policy	Adela Cervantes	1 - Critical	In Progress	Inquiry / Help	Software	Beth Anglin	2026-09-30 01:11:03	admin
INC0000004	2026-09-28 02:55:00	UI Policy Test Incident	System Administrator	4 - Low	In Progress	Inquiry / Help	Impthl	Impthl	2026-09-29 23:40:06	admin
INC0000009	2018-08-30 01:08:35	Unable to access the shared folder	David Miller	4 - Low	New	Inquiry / Help	Impthl	Impthl	2018-12-12 23:30:24	admin
INC0000005	2018-08-31 21:35:21	Email server is down.	David Miller	1 - Critical	New	Software	Impthl	Impthl	2018-12-12 23:18:55	admin
INC0000001	2018-09-11 20:56:26	Unable to post content on a Wiki page	David Miller	3 - Moderate	New	Inquiry / Help	Impthl	Impthl	2018-12-12 23:32:42	admin
INC0000112	2019-07-29 11:48:43	Assessment - ATF Assessor	survey user	5 - Planning	New	Inquiry / Help	Impthl	Impthl	2019-07-29 11:49:28	admin
INC0000111	2019-07-22 14:04:57	ATF Test1	System Administrator	4 - Low	In Progress	Inquiry / Help	Impthl	Impthl	2026-09-29 01:34:39	admin
INC0000001	2021-01-15 13:04:34	ATF TEST2	survey user	5 - Planning	New	Inquiry / Help	Impthl	Impthl	2021-01-21 15:31:42	admin
INC00007002	2018-10-16 22:47:51	Need access to the common drive	David Miller	4 - Low	New	Inquiry / Help	Impthl	Impthl	2018-12-12 23:28:09	admin

Description: The Incident list shows all active incidents. Cells such as State can normally be edited in place, but the onCellEdit Client Script blocks State changes here and tells the user to open the incident instead.

Client Script Record:



Client Script - Prevent state change via list edit

Name: Prevent state change via list edit

Table: Incident [incident]

UI Type: Desktop

Type: onCellEdit

Field name: State

Application: Global

Active: ☒

Inherited: ☐

Global: ☒

Description:

Messages:

Script:

```

1 function onCellEdit(system, table, oldValue, newValue, callback) {
2
3   alert("State cannot be updated using list editing. Please open the Incident.");
4
5   callback(false);
6 }
7
  
```

Description: The Client Script record holds the name, table, UI Type, script type, field name and the JavaScript code, and can be switched on or off with the Active checkbox.

Conclusion:

This project shows how Client Scripts and UI Policies make the ServiceNow Incident form smarter and safer to use. The UI Policy and UI Policy Action control field behaviour instantly, the onChange script reacts as users edit, the onSubmit script checks data before saving, and the onCellEdit script keeps the State field from being changed through list editing. Together they guide users, enforce the rules and reduce incorrect data entry, all without submitting the form. Looking ahead, the same approach can be extended to other tables such as Problem and Change, to the mobile UI, and to more validation rules, and the scripts can be moved into a scoped application so they are easy to package and reuse.